

Yugal — assemble, train, prove

Team lead · RTX 3050 · you own the final model and the stage demo

Your job

Assemble, train, prove, demo

You have the weakest GPU (3050) and the most serial work, so you do almost no extraction. Navya and Akshat feed you embedding roots; you turn them into one model and prove it works.

#	JOB	BLOCKED ON
1	Generate the MMS-TTS set locally (no download)	nothing
2	Build a balanced training manifest	Both extractions
3	Train the production head + 4 ablations	Job 2
4	The Bachchan gate — pass/fail	Job 3
5	Wire the winner into the live UI	Job 4
0	Back up <code>outputs\models</code> — do this first	nothing

JOB 0 — FIVE MINUTES, DO IT BEFORE ANYTHING ELSE

`outputs\models` has been wiped three times in two days, and Navya's merge silently overwrote your original `head.pt` because gitignored files get replaced during a merge without a conflict. Copy the folder somewhere outside the repo now. It is ~7 MB.

```
cp -r outputs/models ~/sonix_models_backup_$(date +%Y%m%d)
```

Job 1

Your own dataset — no download needed

Navya and Akshat own the two downloads. Yours generates locally on the 3050, so it costs no bandwidth and needs nobody:

```
for L in hin mar ben tam tel; do
  python make_indic_spoof.py --lang "$L" --n 160 --out data/indic_spoof
done

python src/extract_embeddings.py --split train --audio-dir data/indic_spoof \
  --out outputs/embeddings_mms_tts --batch 8
python stamp_labels.py --emb-dir outputs/embeddings_mms_tts/train --label 1
python stamp_labels.py --emb-dir outputs/embeddings_mms_tts/train --check
```

It is only ~800 clips from one TTS system per language, so it is the smallest of the three — but it is a third generator family, and diversity is what we are buying.

`run_overnight.sh` does all of this plus the backup, and keeps going if a stage fails.

Job 2

Balance, or the whole exercise fails

Once the new roots land you will have roughly this. The numbers matter more than they look.

ROOT	LABEL	ROWS	FAMILY
embeddings (ASVspoof19)	both	25,380	2019 TTS/VC, English
embeddings_g711	both	25,380	+ phone codec
embeddings_rawboost	both	25,380	+ channel/impulsive
embeddings_rirmusan_*	both	25,380	+ room and noise
embeddings_indicvoices_tr	0 only	~37,000	real Indian speech
embeddings_mlaad_indic Navya	1 only	~20,000	TTS, up to 140 models, Indic
embeddings_indicsynth Akshat	1 only	~16,000	voice conversion, Indic
embeddings_mms_tts you	1 only	~800	a third TTS system, Indic

Check the Indic slice specifically: about **37,000 real** against **37,000 fake**. Roughly balanced — which is the entire point. Right now it is 37,000 real to *zero* fake, and that ratio *is* the bug.

WATCH THE RATIO PER LANGUAGE, NOT JUST OVERALL

If Hindi ends up 20,000 real / 2,000 fake while Tamil is 3,000 / 8,000, the head learns “Hindi = real, Tamil = fake” and you have rebuilt the same bug one level down. Count rows per language before you train and cap the dominant side.

Job 3

Train one production head and four ablations

Each takes minutes once embeddings exist. Run all five — the ablations are what let us say which data actually helped, instead of asserting it.

```
# A. current best, for comparison -- no new data
python src/train.py --emb-root outputs/embeddings \
  --extra-emb-root outputs/embeddings_g711 \
  --extra-emb-root outputs/embeddings_rawboost \
  --extra-emb-root outputs/embeddings_rirmusan_bonafide \
  --extra-emb-root outputs/embeddings_rirmusan_spoof \
  --extra-emb-root outputs/embeddings_indicvoices_tr \
  --out outputs/models/abl_a_no_indic_spoof.pt

# B. + MLAAD only      (does TTS diversity alone fix it?)
# C. + IndicSynth only (does voice conversion alone fix it?)
# D. + MMS-TTS only    (is one extra TTS system enough?)

# E. THE PRODUCTION HEAD -- everything
python src/train.py --emb-root outputs/embeddings \
  --extra-emb-root outputs/embeddings_g711 \
  --extra-emb-root outputs/embeddings_rawboost \
  --extra-emb-root outputs/embeddings_rirmusan_bonafide \
  --extra-emb-root outputs/embeddings_rirmusan_spoof \
  --extra-emb-root outputs/embeddings_indicvoices_tr \
  --extra-emb-root outputs/embeddings_mlaad_indic \
  --extra-emb-root outputs/embeddings_indicsynth \
  --extra-emb-root outputs/embeddings_mms_tts \
  --out outputs/models/head_v3.pt
```

Watch the class balance line `train.py` prints. `pos_weight` should land near `n_neg/n_pos`; if it drifts far from `~1` on the combined set, the ratio is off and you should cap a root before retraining.

Job 4

The gate — this decides everything

One test, four outcomes. Run every head against the Bachchan pair and Akshat's Hindi clips:

```
python bench_clips.py --real <hindi real folder> --fake <hindi clone folder>
python inspect_ckpt.py
```

REAL BACHCHAN	CLONED BACHCHAN	VERDICT
real	fake	Fixed. Ship it, this is the demo.
fake	fake	Still “Indian = fake”. Not enough Indic bonafide, or the spoofs are too clean — augment them.
real	real	Still “Indian = real”. Not enough Indic spoof, or a stamping bug — re-run every <code>--check</code> .
fake	real	Inverted. Something is mislabelled. Check every stamp before anything else.

Then confirm nothing regressed: Navya re-runs DF21 and the Indic holdout against `head_v3`. If DF21 got materially worse, say so and consider dropping a root — do not quietly keep the number that flatters us.

THE HONEST FALLBACK, DECIDED NOW RATHER THAN AT 2AM

If the gate does not pass in time, **do not demo on Indian audio**. The model genuinely works on the English material it was trained and tested on — 1.49% in-domain, $\approx 5.3\%$ on DF21 at 75% coverage, and Akshat's English clips. Demo that, and state the Indic gap as measured, known, and in progress with the datasets named.

A team that says “we tested our own system on Indian speech, found it failed, and here is the fix underway” reads far stronger than one that gets caught by a judge holding a Hindi clip. What we must not do is ship `head_full_ho` on the strength of its 0.02% — we now know what it actually learned.

Job 5

Ship it into the demo

Drop the winner in `outputs/models/` and add it to `realtime/models.py` — it becomes a tab in the sidebar automatically. Current standard setting:

Model	the head that passed Job 4
Amber / Red	0.10 / 0.90
Silence gate	Auto
Smoothing	5-window mean, hysteresis 3-of-5 (fixed)

```
python -m realtime.server --auto-approve --ws-port 8000
streamlit run realtime/live_ui.py
```

What to tell the team

- **Yukti** — every number on every slide has to be re-derived from `head_v3`. The padding-fix slide still stands as-is; the Indic numbers do not. Her `metrics.py` is the canonical one — Navya's older copy should be retired.
- **Suryansh** — regenerate the replay score files once `head_v3` exists, and re-record the backup video. The video is the demo if the laptop dies.
- **Everyone** — nothing goes on a slide from a model we are not shipping.

The situation

What broke, and why

Our detector fails on Indian speech, and we now know exactly why.

Every model up to `head_robust_v2` was trained on ASVspoof 2019 — English studio speech — plus augmentations that vary *rooms, noise and phone channels*. None of them vary *language*. So on 42,082 genuine IndicVoices clips, the baseline called **56% fake** and `robust_v2` called **58.6% fake**. That is why the Modi and Bachchan clips failed.

Navya then added IndicVoices to training and Indian false alarms collapsed to **0.02%**. Real fix? No — a second bug. IndicVoices is **bonafide-only**: every one of its $\sim 37,000$ rows is labelled real. So the only thing the model could learn from Indian audio was “*Indian phonetics* $\rightarrow$ *real*”. Tested on a cloned Amitabh Bachchan clip, `head_full_ho` calls it **real**.

MODEL	REAL BACHCHAN	CLONED BACHCHAN	WHAT IT ACTUALLY LEARNED
head_robust_v2	fake ✗	fake ✓	“Indian audio = fake”
head_full_ho	real ✓	real ✗	“Indian audio = real”

Neither is detecting anything. Each gets one column right by accident.

THE ROOT CAUSE, IN ONE SENTENCE

As long as every Indian-language sample in training is labelled real, the language itself is a shortcut to “real”, and a cloned Indian voice walks straight through. No threshold, augmentation or calibration fixes this.

The fix

Put Indian languages on **both sides of the label**, from **several different synthesis families**. Then the model cannot use language or accent as a cue — it has to find real synthesis artefacts or nothing.

DATASET	SYNTHESIS FAMILY	OWNER
Indic-CodecFake	Neural audio codecs — 8 of them (DAC, Encodec, SoundStream, SpeechTokenizer, FunCodec, AudioDec, SNAC, MIMI)	Navya
MLAAD (Indic slice)	Text-to-speech — up to 140 models	Akshat
IndicSynth	Voice conversion (freevc24)	Akshat
IndicVoices have it	Real human speech (bonafide side)	Navya

Three *families*, not just more hours. A head trained on one generator learns that generator — which is precisely the trap we fell into with English-only ASVspoof 2019. Generator diversity is what buys generalisation.

How we train

One model, many ablations

Two questions came up, so here they are answered for everyone.

Is the new model built from scratch, or on top of the latest one?

Neither, exactly — and this matters. Our pipeline has two stages:

STAGE	COST	REUSABLE?
audio → embeddings (frozen wav2vec2 XLS-R, 300M params)	GPU, hours	Yes — forever, per dataset
embeddings → head (MLP, 262,657 params)	CPU/GPU, minutes	Cheap to redo

So we never “continue training” `head_full_ho`. We retrain a **fresh head on the union of every embedding root** — the old ones plus the new ones. That is what `--extra-emb-root` does, and it stacks (it uses `action="append"`, verified).

Fine-tuning the existing head on only the new data would be actively wrong: it would forget the English attacks it can currently catch.

One model or several?

One production model, plus four throwaway ablation heads for the results slide. Once the embeddings exist, each head takes minutes, so the ablations are nearly free — and they are what let us say “*we measured which data helped*” instead of “*we threw everything in and hoped*”.

The expensive, parallel part is extraction. That is why it is split three ways across our GPUs. Training is serial and Yugal does it.

THE RULE THAT MAKES OR BREAKS THIS

Cap clips per (language × generator). If one TTS model contributes 40,000 clips and another contributes 200, the head learns to detect the loud one. Every extraction step below has a cap flag. Use it.

Hard rules

What must never enter training

These are test sets. Training on any of them destroys the numbers that prove we generalise, and a judge who spotted it would be right to dismiss the project.

- **ASVspoof 2021 DF** — our cross-dataset headline
- **In-the-Wild** — the honest generalisation test, named on slide 3
- **The IndicVoices holdout** (`embeddings_indicvoices_ho`) — 12% held out by recording
- **Akshat's 20 labelled clips** and **the Bachchan pair** — our real-world evidence

Everything else — ASVspoof 2019 train, the augmented copies, IndicVoices train split, Indic-CodecFake, MLAAD, IndicSynth — is fair training data.

SECOND HARD RULE – LABEL STAMPING

`extract_embeddings.py --audio-dir` marks **everything label 0 (real)** by default. Every spoof folder must be stamped `--label 1` afterwards, and verified with `--check`. Skip it and your fakes train as genuine — a silent, catastrophic bug that nothing warns you about.

`stamp_labels.py` was broken until 2 Sept (it never wrote a label successfully). Make sure you have pulled since then, and always read the `--check` output rather than assuming.

Sections after your own tasks are identical in all three handbooks, so nobody has to read someone else's. Datasets: Indic-CodecFake (helixometry.github.io/IndicFake) · MLAAD (deepfake-total.com/mlaad) · IndicSynth (huggingface.co/datasets/vdivyasharma/IndicSynth). When in doubt, stop and ask Yugal — a wrong number is far worse than a missing one.